

Isabelle Formalization of Greedy Algorithms for Cardinality-Constrained Submodular Maximization

Feier Lyu

April 18, 2026

Abstract

We formalize in Isabelle/HOL the classical approximation guarantee for monotone non-negative submodular maximization under a cardinality constraint on a finite ground set. The main result is the Nemhauser–Wolsey bound for deterministic greedy: after k steps, the greedy solution achieves the finite-step guarantee $1 - (1 - 1/k)^k$, and hence also the standard $(1 - 1/e)$ approximation ratio. The development also includes a verified stateful lazy greedy refinement and proves that it satisfies the same approximation guarantee.

Contents

1	Greedy construction	3
1.1	Preliminaries on finite maximizers	4
2	Concrete argmax oracle for marginal gain	4
3	Lazy (accelerated) selection via upper bounds	8
4	Stateful Lazy Greedy with cached upper bounds	10
4.1	State (selected set + cached upper bounds)	10
4.2	Inner lazy selection that returns updated ub	11
4.3	ub validity carried over to the next outer iteration	11
4.4	One outer step and the full stateful algorithm	12
4.5	Main invariants: subset property and validity on the remaining set	12
4.6	Greedy-step correctness (each chosen x is a true argmax of gain)	13
5	Greedy approximation for monotone submodular maximization	14

6	Submodular setting	14
6.1	Non-emptiness lemmas	14
6.2	Submodular calculus	15
6.3	Main averaging lemma	15
6.4	Existence of maximal element in finite sets	15
6.5	OPT as a maximizer over feasible sets	16
6.6	Gap sequence	16
6.7	Non-negativity of OPT and approximation ratio	18
6.8	Corollaries	18

7 Step-spec corollary 19

```

theory Submodular_Base
  imports Complex_Main
begin

locale Submodular_Func =
  fixes V :: "'a set" and f :: "'a set  $\Rightarrow$  real"
  assumes finite_V: "finite V"
    and monotone_f: " $\bigwedge S T. S \subseteq T \Rightarrow T \subseteq V \Rightarrow f S \leq f T$ "
    and submodular_f:
      " $\bigwedge S T. S \subseteq V \Rightarrow T \subseteq V \Rightarrow f (S \cup T) + f (S \cap T) \leq f S + f T$ "
  and f_empty: "f {} = 0"
begin

  Marginal gain of adding a single element to a set.

definition gain :: "'a set  $\Rightarrow$  'a  $\Rightarrow$  real" where
  "gain S e = f (S  $\cup$  {e}) - f S"

lemma f_nonneg:
  assumes "S  $\subseteq$  V"
  shows "0  $\leq$  f S"
<proof>

lemma monotone_on_PowV:
  shows "monotone_on (Pow V) ( $\subseteq$ ) ( $\leq$ ) f"
<proof>

lemma monotone_f_from_monotone_on_PowV:
  assumes mono: "monotone_on (Pow V) ( $\subseteq$ ) ( $\leq$ ) f"
  assumes "S  $\subseteq$  T" "T  $\subseteq$  V"
  shows "f S  $\leq$  f T"
<proof>

lemma gain_nonneg:
  assumes "S  $\subseteq$  V" and "x  $\in$  V - S"
  shows "0  $\leq$  gain S x"
<proof>

```

Diminishing returns for single-element marginal gains.

```

lemma gain_decreasing:
  assumes " $S \subseteq T$ " " $T \subseteq V$ " " $x \in V$ " " $x \notin T$ "
  shows " $\text{gain } S \ x \geq \text{gain } T \ x$ "
   $\langle \text{proof} \rangle$ 

end

locale Cardinality_Constraint = Submodular_Func +
  fixes  $k :: \text{nat}$ 
  assumes  $k\_le\_cardV$ : " $k \leq \text{card } V$ "
begin

definition feasible :: "'a set  $\Rightarrow$  bool" where
  " $\text{feasible } S \longleftrightarrow S \subseteq V \wedge \text{card } S \leq k$ "

lemma feasibleI:
  assumes " $S \subseteq V$ " " $\text{card } S \leq k$ "
  shows " $\text{feasible } S$ "
   $\langle \text{proof} \rangle$ 

lemma feasibleD:
  assumes " $\text{feasible } S$ "
  shows " $S \subseteq V$ " " $\text{card } S \leq k$ "
   $\langle \text{proof} \rangle$ 

lemma feasible_empty[simp]: " $\text{feasible } \{\}$ "
   $\langle \text{proof} \rangle$ 

lemma feasible_family_nonempty: " $\text{Collect feasible} \neq \{\}$ "
   $\langle \text{proof} \rangle$ 

lemma finite_feasible_family: " $\text{finite } \{S. \text{feasible } S\}$ "
   $\langle \text{proof} \rangle$ 

end

end

theory Greedy_Submodular_Construct
  imports "../Core/Submodular_Base"
begin

```

1 Greedy construction

This theory sets up the greedy construction for monotone submodular maximization under a cardinality constraint. We fix a finite ground set V , a budget k , and a non-negative monotone submodular function f over subsets of V . The greedy sequence starts from the empty set and repeatedly adds an

element with the largest marginal gain (ties broken arbitrarily).

Locale *Greedy_Setup* encapsulates this standard setting. All subsequent definitions and lemmas are expressed relative to a fixed ground set V , budget k , and set function f satisfying the usual assumptions (finiteness, non-negativity, monotonicity, and submodularity).

1.1 Preliminaries on finite maximizers

Finite arg-max via the standard maximality predicate.

```
lemma finite_is_arg_max_in:
  fixes g :: "'a ⇒ 'b::linorder"
  assumes fin: "finite A" and ne: "A ≠ {}"
  shows "∃ x∈A. is_arg_max g (λx. x ∈ A) x"
  <proof>
```

```
lemma is_arg_maxD_le:
  fixes f :: "'b ⇒ 'a::linorder"
  assumes "is_arg_max f P x" "P y"
  shows "f y ≤ f x"
  <proof>
```

Abstract setup for the greedy algorithm: a finite ground set V , a budget k , and a non-negative monotone submodular function f with $f \equiv 0$.

This theory focuses on the greedy construction and basic structural properties, without yet proving approximation guarantees.

2 Concrete argmax oracle for marginal gain

```
context Submodular_Func
begin
```

```
definition argmax_gain_some :: "'a set ⇒ 'a set ⇒ 'a" where
  "argmax_gain_some S A =
    (SOME x. x ∈ A ∧ is_arg_max (gain S) (λy. y ∈ A) x)"
```

```
lemma argmax_gain_some_mem:
  assumes fin: "finite A" and ne: "A ≠ {}"
  shows "argmax_gain_some S A ∈ A"
  <proof>
```

```
lemma argmax_gain_some_max:
  assumes fin: "finite A" and ne: "A ≠ {}" and yA: "y ∈ A"
  shows "gain S y ≤ gain S (argmax_gain_some S A)"
  <proof>
```

```
end
```

```

locale Greedy_Setup = Cardinality_Constraint V f k
  for V :: "'a set" and f :: "'a set  $\Rightarrow$  real" and k :: nat +
  fixes argmax_gain :: "'a set  $\Rightarrow$  'a set  $\Rightarrow$  'a"
  assumes argmax_gain_mem:
    "finite A  $\Rightarrow$  A  $\neq$  {}  $\Rightarrow$  argmax_gain S A  $\in$  A"
  assumes argmax_gain_max:
    "finite A  $\Rightarrow$  A  $\neq$  {}  $\Rightarrow$   $\forall y \in A. \text{gain } S \ y \leq \text{gain } S \ (\text{argmax\_gain } S \ A)"$ 
begin

```

Greedy construction: start from {} and, as long as there are remaining elements, add one with maximum marginal gain.

```

fun greedy_set :: "nat  $\Rightarrow$  'a set" where
  "greedy_set 0 = {}"
| "greedy_set (Suc i) =
  (let S = greedy_set i in
   if V - S = {} then S else insert (argmax_gain S (V - S)) S)"

```

Greedy sets are always subsets of the ground set V.

```

lemma greedy_subset_V: "greedy_set i  $\subseteq$  V"
<proof>

```

If a genuinely new element is inserted, the cardinality increases by one.

```

lemma greedy_card_step:
  assumes ne: "V - greedy_set i  $\neq$  {}"
  shows "card (greedy_set (Suc i)) = card (greedy_set i) + 1"
<proof>

```

If the remainder is empty, the greedy set stays unchanged.

```

lemma greedy_card_idle:
  assumes "V - greedy_set i = {}"
  shows "card (greedy_set (Suc i)) = card (greedy_set i)"
<proof>

```

State transition: when the remainder is non-empty, S evolves by adding the arg-max element.

```

lemma state_transition_nonempty:
  assumes "0 < i" and "V - greedy_set (i - 1)  $\neq$  {}"
  shows "greedy_set i =
    greedy_set (i - 1)
     $\cup$  {argmax_gain (greedy_set (i - 1)) (V - greedy_set (i - 1))}"
<proof>

```

At most one new element is added in each greedy step.

```

lemma card_greedy_le_i: "card (greedy_set i)  $\leq$  i"
<proof>

```

If $i \leq k$ then $\text{card } (\text{greedy_set } i) \leq k$ (used later in the gap bound).

```

lemma card_greedy_le_k:
  assumes "i ≤ k"
  shows "card (greedy_set i) ≤ k"
  <proof>

```

If $\text{card } (\text{greedy_set } t) < \text{card } V$, then the remainder $V - \text{greedy_set } t$ is non-empty.

```

lemma remainder_nonempty_if_card_ltV:
  assumes "card (greedy_set t) < card V"
  shows "V - greedy_set t ≠ {}"
  <proof>

```

Under $k \leq \text{card } V$, the greedy transition up to step k always adds a new element.

```

lemma state_transition_upto_k:
  assumes "0 < i" "i ≤ k"
  shows "greedy_set i =
        greedy_set (i - 1)
        ∪ {argmax_gain (greedy_set (i - 1)) (V - greedy_set (i - 1))}"
  <proof>

```

Intermediate greedy states as a list $[S, \dots, S]$.

```

definition greedy_sequence :: "nat ⇒ 'a set list" where
  "greedy_sequence n = map greedy_set [0.. $\text{Suc } n$ ]"

```

Indexing lemma for the sequence representation.

```

lemma greedy_sequence_nth[simp]:
  assumes "i ≤ n"
  shows "greedy_sequence n ! i = greedy_set i"
  <proof>

```

Every greedy state is finite.

```

lemma greedy_set_finite [simp]: "finite (greedy_set i)"
  <proof>

```

One-step monotonicity: $S \subseteq S$.

```

lemma greedy_mono_Suc: "greedy_set i ⊆ greedy_set (Suc i)"
  <proof>

```

Chain monotonicity: if $i \leq j$ then $S \subseteq S$.

```

lemma greedy_chain_mono:
  assumes "i ≤ j"
  shows "greedy_set i ⊆ greedy_set j"
  <proof>

```

Cardinality is non-decreasing along the greedy sequence.

```

lemma greedy_card_mono:
  "i ≤ j ⇒ card (greedy_set i) ≤ card (greedy_set j)"

```

$\langle \text{proof} \rangle$

A compact bound in one line: $\text{card } S \leq \min i (\text{card } V)$ for all i .

```
lemma greedy_card_min:
  "card (greedy_set i) ≤ min i (card V)"
  <proof>
```

Length and endpoints of the intermediate sequence.

```
lemma greedy_sequence_length [simp]:
  "length (greedy_sequence n) = Suc n"
  <proof>
```

```
lemma greedy_sequence_0 [simp]:
  "greedy_sequence n ! 0 = {}"
  <proof>
```

```
lemma greedy_sequence_last [simp]:
  "greedy_sequence n ! n = greedy_set n"
  <proof>
```

At a non-empty remainder, the chosen element is new and lies in $V - S$.

```
lemma chosen_in_remainder_nonempty:
  assumes rem_ne: "V - greedy_set i ≠ {}"
  defines x_def: "x ≡ argmax_gain (greedy_set i) (V - greedy_set i)"
  shows "x ∈ V - greedy_set i" "x ∉ greedy_set i"
  <proof>
```

Increment shape at a non-empty step: S is obtained by inserting the arg-max element into S . This is often useful in counting arguments.

```
lemma greedy_increment_nonempty [simp]:
  assumes rem_ne: "V - greedy_set i ≠ {}"
  shows "greedy_set (Suc i) =
        insert (argmax_gain (greedy_set i) (V - greedy_set i)) (greedy_set
i)"
  <proof>
```

One-step update of the objective value along the greedy sequence.

```
lemma greedy_step_f_eq:
  assumes "V - greedy_set i ≠ {}"
  shows
    "f (greedy_set (Suc i)) =
     f (greedy_set i) +
     gain (greedy_set i)
     (argmax_gain (greedy_set i) (V - greedy_set i))"
  <proof>
```

end

context Cardinality_Constraint

begin

interpretation *Greedy_Concrete*: *Greedy_Setup* *V f k argmax_gain_some*
<proof>

end

end

theory *Lazy_Greedy_Oracle*

imports *"Greedy_Submodular_Construct"*

begin

Auxiliary oracle-style lazy selection primitives based on cached upper bounds. This theory is not the main theorem-facing LazyGreedy development. Its role is to provide backend selection machinery reused by the stateful LazyGreedy line.

3 Lazy (accelerated) selection via upper bounds

context *Submodular_Func*

begin

definition *ub_valid* :: "'a set $\Rightarrow$ 'a set $\Rightarrow$ ('a $\Rightarrow$ real) $\Rightarrow$ bool" **where**
"*ub_valid* *S A ub* $\longleftrightarrow (\forall x \in A. \text{gain } S \ x \leq \text{ub } x)$ "

definition *untight* :: "'a set $\Rightarrow$ 'a set $\Rightarrow$ ('a $\Rightarrow$ real) $\Rightarrow$ 'a set" **where**
"*untight* *S A ub* = {*x* $\in$ *A*. *ub* *x* > *gain* *S* *x*}"

definition *tighten* :: "'a set $\Rightarrow$ ('a $\Rightarrow$ real) $\Rightarrow$ 'a $\Rightarrow$ ('a $\Rightarrow$ real)" **where**
"*tighten* *S ub* *x* = *ub*(*x* := *gain* *S* *x*)"

definition *pick_ub_some* :: "'a set $\Rightarrow$ ('a $\Rightarrow$ real) $\Rightarrow$ 'a" **where**
"*pick_ub_some* *A ub* = (SOME *x*. *x* $\in$ *A* $\wedge$ *is_arg_max* *ub* ($\lambda y. y \in A$) *x*)"

lemma *pick_ub_some_mem*:

assumes *finA*: "finite *A*" **and** *neA*: "*A* $\neq$ {}"

shows "*pick_ub_some* *A ub* $\in$ *A*"

<proof>

lemma *pick_ub_some_max*:

assumes *finA*: "finite *A*" **and** *neA*: "*A* $\neq$ {}" **and** *yA*: "*y* $\in$ *A*"

shows "*ub* *y* $\leq$ *ub* (*pick_ub_some* *A ub*)"

<proof>

fun *lazy_argmax_gain_fuel* ::

"nat $\Rightarrow$ 'a set $\Rightarrow$ 'a set $\Rightarrow$ ('a $\Rightarrow$ real) $\Rightarrow$ 'a"

where

"*lazy_argmax_gain_fuel* 0 *S A ub* = *pick_ub_some* *A ub*"

```

| "lazy_argmax_gain_fuel (Suc n) S A ub =
  (let x = pick_ub_some A ub in
   if ub x = gain S x then x
   else lazy_argmax_gain_fuel n S A (tighten S ub x))"

definition lazy_argmax_gain :: "'a set  $\Rightarrow$  'a set  $\Rightarrow$  ('a  $\Rightarrow$  real)  $\Rightarrow$  'a"
where
  "lazy_argmax_gain S A ub = lazy_argmax_gain_fuel (card A) S A ub"

lemma ub_valid_tighten:
  assumes ubv: "ub_valid S A ub"
  shows "ub_valid S A (tighten S ub x)"
  <proof>

lemma untight_tighten:
  assumes xA: "x  $\in$  A" and gt: "ub x > gain S x"
  shows "untight S A (tighten S ub x) = untight S A ub - {x}"
  <proof>

lemma finite_untight:
  assumes finA: "finite A"
  shows "finite (untight S A ub)"
  <proof>

lemma lazy_argmax_gain_fuel_max:
  assumes finA: "finite A" and neA: "A  $\neq$  {}"
  shows "ub_valid S A ub  $\implies$  card (untight S A ub)  $\leq$  n  $\implies$ 
     $\forall y \in A. \text{gain } S \ y \leq \text{gain } S \ (\text{lazy\_argmax\_gain\_fuel } n \ S \ A \ ub)"$ 
  <proof>

lemma lazy_argmax_gain_fuel_mem:
  assumes finA: "finite A" and neA: "A  $\neq$  {}"
  shows "lazy_argmax_gain_fuel n S A ub  $\in$  A"
  <proof>

lemma lazy_argmax_gain_mem:
  assumes finA: "finite A" and neA: "A  $\neq$  {}"
  shows "lazy_argmax_gain S A ub  $\in$  A"
  <proof>

lemma lazy_argmax_gain_max:
  assumes finA: "finite A" and neA: "A  $\neq$  {}"
  and ubv: "ub_valid S A ub"
  shows " $\forall y \in A. \text{gain } S \ y \leq \text{gain } S \ (\text{lazy\_argmax\_gain } S \ A \ ub)"$ 
  <proof>

end

```

```

context Cardinality_Constraint
begin

definition argmax_gain_lazy :: "'a set  $\Rightarrow$  'a set  $\Rightarrow$  'a" where
  "argmax_gain_lazy S A = lazy_argmax_gain S A (gain S)"

lemma ub_valid_gain [simp]:
  "ub_valid S A (gain S)"
  <proof>

lemma argmax_gain_lazy_mem:
  assumes "finite A" and "A  $\neq$  {}"
  shows "argmax_gain_lazy S A  $\in$  A"
  <proof>

lemma argmax_gain_lazy_max:
  assumes "finite A" and "A  $\neq$  {}"
  shows " $\forall y \in A. \text{gain } S \ y \leq \text{gain } S \ (\text{argmax\_gain\_lazy } S \ A)$ "
  <proof>

end

end
theory Lazy_Greedy_Stateful
  imports Lazy_Greedy_Oracle
begin

```

4 Stateful Lazy Greedy with cached upper bounds

```

record 'a lg_state =
  Sg  :: "'a set"
  ubg :: "'a  $\Rightarrow$  real"

```

```

context Cardinality_Constraint
begin

```

4.1 State (selected set + cached upper bounds)

```

definition init_ub :: "'a  $\Rightarrow$  real" where
  "init_ub x = gain {} x"

```

```

definition init_state :: "'a lg_state" where
  "init_state = ( $\lambda$  Sg = {}, ubg = init_ub  $\lambda$ )"

```

```

definition remaining :: "'a lg_state  $\Rightarrow$  'a set" where
  "remaining st = V - Sg st"

```

4.2 Inner lazy selection that returns updated ub

```

fun lazy_argmax_gain_fuel_state ::
  "nat  $\Rightarrow$  'a set  $\Rightarrow$  'a set  $\Rightarrow$  ('a  $\Rightarrow$  real)  $\Rightarrow$  ('a  $\times$  ('a  $\Rightarrow$  real))"
where
  "lazy_argmax_gain_fuel_state 0 S A ub =
    (let x = pick_ub_some A ub in (x, ub))"
| "lazy_argmax_gain_fuel_state (Suc n) S A ub =
    (let x = pick_ub_some A ub in
      if ub x = gain S x then (x, ub)
      else lazy_argmax_gain_fuel_state n S A (tighten S ub x))"

definition lazy_select :: "'a set  $\Rightarrow$  'a set  $\Rightarrow$  ('a  $\Rightarrow$  real)  $\Rightarrow$  ('a  $\times$  ('a
 $\Rightarrow$  real))" where
  "lazy_select S A ub = lazy_argmax_gain_fuel_state (card A) S A ub"

lemma lazy_argmax_gain_fuel_state_fst:
  "fst (lazy_argmax_gain_fuel_state n S A ub) = lazy_argmax_gain_fuel
n S A ub"
<proof>

lemma lazy_select_fst:
  "fst (lazy_select S A ub) = lazy_argmax_gain S A ub"
<proof>

lemma lazy_argmax_gain_fuel_state_ub_valid:
  assumes ubv: "ub_valid S A ub"
  shows "ub_valid S A (snd (lazy_argmax_gain_fuel_state n S A ub))"
<proof>

lemma lazy_select_ub_valid:
  assumes ubv: "ub_valid S A ub"
  shows "ub_valid S A (snd (lazy_select S A ub))"
<proof>

```

4.3 ub validity carried over to the next outer iteration

```

lemma ub_valid_init:
  "ub_valid {} V init_ub"
<proof>

lemma ub_valid_after_insert:
  assumes ubv: "ub_valid S (V - S) ub"
  and Ssub: "S  $\subseteq$  V"
  and x_in: "x  $\in$  V - S"
  shows "ub_valid (insert x S) (V - insert x S) ub"
<proof>

```

4.4 One outer step and the full stateful algorithm

definition lazy_step :: "'a lg_state $\Rightarrow$ 'a lg_state" where

```
"lazy_step st =
  (let S = Sg st;
    A = remaining st;
    ub = ubg st
  in if A = {} then st
    else
      (let p = lazy_select S A ub;
        x = fst p;
        ub' = snd p
      in ( $\mid$  Sg = insert x S, ubg = ub'  $\mid$ )))"
```

lemma lazy_step_nonempty_Sg:

assumes rem_ne: "remaining st $\neq$ {}"

shows

```
"Sg (lazy_step st) =
  insert (fst (lazy_select (Sg st) (remaining st) (ubg st))) (Sg st)"
<proof>
```

lemma lazy_step_nonempty_ubg:

assumes rem_ne: "remaining st $\neq$ {}"

defines "p $\equiv$ lazy_select (Sg st) (remaining st) (ubg st)"

shows "ubg (lazy_step st) = snd p"

<proof>

fun lazy_state :: "nat $\Rightarrow$ 'a lg_state" where

"lazy_state 0 = init_state"

| "lazy_state (Suc i) = lazy_step (lazy_state i)"

definition lazy_set :: "nat $\Rightarrow$ 'a set" where

"lazy_set i = Sg (lazy_state i)"

4.5 Main invariants: subset property and validity on the remaining set

lemma lazy_step_idle:

assumes "remaining st = {}"

shows "lazy_step st = st"

<proof>

lemma lazy_state_subset_V:

"Sg (lazy_state i) $\subseteq$ V"

<proof>

lemma lazy_state_ub_valid:

"ub_valid (Sg (lazy_state i)) (remaining (lazy_state i)) (ubg (lazy_state i))"

<proof>

4.6 Greedy-step correctness (each chosen x is a true argmax of gain)

```
lemma lazy_step_is_argmax:
  assumes rem_ne: "remaining st  $\neq$  {}"
    and ubv: "ub_valid (Sg st) (remaining st) (ubg st)"
    and finA: "finite (remaining st)"
  defines "p  $\equiv$  lazy_select (Sg st) (remaining st) (ubg st)"
  defines "x  $\equiv$  fst p"
  shows " $\forall y \in \text{remaining st. gain (Sg st) y} \leq \text{gain (Sg st) x}$ "
<proof>
```

end

end

theory Greedy_Step_Spec

imports

"../Algorithms/Greedy_Submodular_Construct"

begin

Step-spec interface for greedy-style algorithms.

Any oracle that, on every finite non-empty candidate set, returns an element with maximal marginal gain can be interpreted as an instance of the greedy setup locale.

```
locale Greedy_Step_Oracle =
  Cardinality_Constraint V f k
  for V :: "'a set" and f :: "'a set  $\Rightarrow$  real" and k :: nat +
  fixes select :: "'a set  $\Rightarrow$  'a set  $\Rightarrow$  'a"
  assumes select_mem:
    "finite A  $\Rightarrow$  A  $\neq$  {}  $\Rightarrow$  select S A  $\in$  A"
  assumes select_max:
    "finite A  $\Rightarrow$  A  $\neq$  {}  $\Rightarrow$  ( $\forall y \in A. \text{gain S y} \leq \text{gain S (select S A)}$ )"
begin
```

sublocale Greedy_Setup V f k select

<proof>

end

end

theory Greedy_Submodular_Approx

imports

Greedy_Step_Spec

begin

This theory derives analytic bounds for the coefficient $1 - (1 - 1/k)^k$ appearing in the Nemhauser–Wolsey approximation ratio. In particular we

show that it is bounded below by $1 - 1/\exp 1$.

First, we relate the discrete quantity $(1 - 1/k)^k$ to the exponential function via a standard limit inequality.

```
lemma pow_one_minus_inv_le_exp_neg1:
  fixes k :: nat
  assumes "k ≥ 1"
  shows "(1 - 1 / real k) ^ k ≤ exp (-1 :: real)"
  <proof>
```

As a corollary, we obtain a uniform lower bound $1 - (1 - 1/k)^k \geq 1 - 1/\exp 1$ for all $k \geq 1$.

```
lemma coeff_ge_1_minus_inv_exp:
  fixes k :: nat
  assumes "k ≥ 1"
  shows "1 - (1 - 1 / real k) ^ k ≥ 1 - 1 / exp 1"
  <proof>
```

```
context Greedy_Setup
begin
```

5 Greedy approximation for monotone submodular maximization

In this section we formalize the classical Nemhauser–Wolsey guarantee: for a non-negative, monotone, submodular function on a finite ground set, the greedy algorithm that selects k elements achieves at least $1 - (1 - 1/k)^k$ times the optimal value. Combining this with the analytic bound above yields the familiar $1 - 1/e$ approximation factor.

Elementary algebraic identity used in the gap recurrence.

```
lemma one_minus_inv_times:
  fixes x :: real
  shows "(1 - 1 / real k) * x = x - x / real k"
  <proof>
```

6 Submodular setting

6.1 Non-emptiness lemmas

We use two basic non-emptiness facts.

First, if S is a subset of V and $\text{card } S < k$, then the candidate set $V - S$ is non-empty.

Second, if both S and Opt are subsets of V and $f S < f \text{ Opt}$, then the gap set $\text{Opt} - S$ is non-empty.

```

lemma nonempty_candidates:
  assumes "S ⊆ V" "card S < k"
  shows "V - S ≠ {}"
⟨proof⟩

lemma nonempty_gap:
  assumes "S ⊆ V" "Opt ⊆ V" "f S < f Opt"
  shows "Opt - S ≠ {}"
⟨proof⟩

```

6.2 Submodular calculus

We next derive a diminishing-returns inequality for marginal gains and a telescoping upper bound over a finite disjoint set.

6.3 Main averaging lemma

We now establish the averaging argument underlying the greedy guarantee. Intuitively, as long as $|S| < k$ and $|Opt| \leq k$, there exists some element in $V - S$ whose marginal gain is at least the average contribution of elements in an optimal solution.

Submodular telescoping: sum of marginals upper-bounds the joint gain.

```

lemma submod_sum_upper:
  assumes "finite A" "A ⊆ V" "S ⊆ V" "A ∩ S = {}"
  shows "f (S ∪ A) - f S ≤ (∑ x∈A. gain S x)"
⟨proof⟩

```

Average marginal bound against any feasible Opt with $|Opt| \leq k$: there exists an element $e \in V - S$ such that $\text{gain } S \ e \geq (f \text{ Opt} - f S) / \text{real } k$.

```

lemma marginal_gain_lower_bound:
  fixes Opt S :: "'a set"
  assumes S_sub: "S ⊆ V"
    and O_sub: "Opt ⊆ V"
    and cardS_lt_k: "card S < k"
    and cardO_le_k: "card Opt ≤ k"
  shows "∃ e∈V - S. gain S e ≥ (f Opt - f S) / real k"
⟨proof⟩

```

6.4 Existence of maximal element in finite sets

Set-function version of the finite arg-max lemma: there is a maximizer of f over any finite non-empty family of sets.

```

lemma finite_has_maximal_is_arg:
  assumes "finite A" "A ≠ {}"
  shows "∃ x∈A. is_arg_max f (λx. x ∈ A) x"

```

$\langle proof \rangle$

corollary *finite_has_maximal*:
 assumes "finite A" "A $\neq \{\}$ "
 shows " $\exists x \in A. \forall y \in A. f\ y \leq f\ x$ "
 $\langle proof \rangle$

6.5 OPT as a maximizer over feasible sets

Using the choice operator *SOME*, we select a canonical optimal set *OPT_set* and define *OPT_k* = *f OPT_set*. All subsequent bounds are expressed in terms of this quantity.

definition *OPT_set* :: "'a set" where
 "OPT_set =
 (SOME X. feasible X $\wedge$ ($\forall Y$. feasible Y $\longrightarrow$ f Y $\leq$ f X))"

lemma *exists_max_feasible*:
 " $\exists X$. feasible X $\wedge$ ($\forall Y$. feasible Y $\longrightarrow$ f Y $\leq$ f X)"
 $\langle proof \rangle$

lemma *OPT_set_props*:
 shows *OPT_set_in*: "feasible OPT_set"
 and *OPT_set_max*: " $\forall Y$. feasible Y $\longrightarrow$ f Y $\leq$ f OPT_set"
 $\langle proof \rangle$

definition *OPT_k* :: real where
 "OPT_k = f OPT_set"

lemma *exists_opt_set*:
 " $\exists X$. feasible X $\wedge$ f X = OPT_k"
 $\langle proof \rangle$

lemma *OPT_k_upper_bound*:
 assumes "feasible S"
 shows "f S $\leq$ OPT_k"
 $\langle proof \rangle$

6.6 Gap sequence

We introduce the gap sequence *gap i* = *OPT_k* - *f (greedy_set i)* and show that it satisfies a simple linear recurrence under the greedy update.

definition *gap* :: "nat $\Rightarrow$ real" where
 "gap i = OPT_k - f (greedy_set i)"

One-step inequality: the marginal gain of the greedy choice lower-bounds the average improvement towards *OPT_k*.

lemma *greedy_step_ineq*:
 assumes "i < k"

```

    and S_sub: "greedy_set i  $\subseteq$  V"
    and R_nonempty: "V - greedy_set i  $\neq$  {}"
  shows "gain (greedy_set i)
        (argmax_gain (greedy_set i) (V - greedy_set i))
         $\geq$  (OPT_k - f (greedy_set i)) / real k"
<proof>

```

Greedy sets are feasible whenever their size is at most k .

```

lemma greedy_set_feasible:
  assumes S_sub: "greedy_set i  $\subseteq$  V"
    and card_le_i: "card (greedy_set i)  $\leq$  i"
    and i_le_k: "i  $\leq$  k"
  shows "feasible (greedy_set i)"
<proof>

```

```

corollary greedy_feasible:
  assumes "i  $\leq$  k"
  shows "feasible (greedy_set i)"
<proof>

```

The gap is non-negative along the greedy sequence.

```

lemma gap_nonneg:
  assumes S_sub: "greedy_set i  $\subseteq$  V"
    and card_le_i: "card (greedy_set i)  $\leq$  i"
    and i_le_k: "i  $\leq$  k"
  shows "0  $\leq$  gap i"
<proof>

```

```

corollary greedy_gap_nonneg:
  assumes "i  $\leq$  k"
  shows "0  $\leq$  gap i"
<proof>

```

```

corollary greedy_remainder_nonempty:
  assumes "i < k"
  shows "V - greedy_set i  $\neq$  {}"
<proof>

```

Gap recurrence: each step reduces the gap by at least a $1/k$ fraction.

```

lemma gap_step_diff:
  assumes "i < k"
    and S_sub: "greedy_set i  $\subseteq$  V"
    and R_nonempty: "V - greedy_set i  $\neq$  {}"
  shows "gap (Suc i)  $\leq$  gap i - gap i / real k"
<proof>

```

In multiplicative form, the gap shrinks by a factor of at most $1 - 1/\text{real } k$ per step.

```

lemma gap_step:

```

```

    assumes "i < k"
    and "greedy_set i ⊆ V"
    and "V - greedy_set i ≠ {}"
    shows "gap (Suc i) ≤ (1 - 1 / real k) * gap i"
  <proof>

```

```

lemma gap_0[simp]: "gap 0 = OPT_k"
  <proof>

```

Geometric decay of the gap: after i greedy steps the remaining gap is bounded by $(1 - 1/\text{real } k)^i * \text{OPT}_k$.

```

lemma gap_geometric:
  assumes k_pos: "k > 0"
  and i_le_k: "i ≤ k"
  shows "gap i ≤ (1 - 1 / real k) ^ i * OPT_k"
  <proof>

```

As a consequence, the value of the greedy solution after k steps is at least $(1 - (1 - 1/\text{real } k)^k) * \text{OPT}_k$.

```

lemma greedy_sequence_bound:
  assumes k_pos: "k > 0"
  shows "f (greedy_set k) ≥ (1 - (1 - 1 / real k) ^ k) * OPT_k"
  <proof>

```

6.7 Non-negativity of OPT and approximation ratio

OPT_k is non-negative because $f \ \{\} = 0$ is a feasible value.

```

lemma OPT_k_nonneg: "0 ≤ OPT_k"
  <proof>

```

Combining the discrete bound with the analytic inequality for $1 - (1 - 1/k)^k$ yields the standard $1 - 1/e$ approximation factor.

```

theorem greedy_approximation:
  assumes k_pos: "k > 0"
  shows "f (greedy_set k) ≥ (1 - 1 / exp 1) * OPT_k"
  <proof>

```

6.8 Corollaries

Define the approximation ratio of the greedy algorithm for a given k (with the convention that the ratio is 1 when $\text{OPT}_k = 0$), and show that it is always at least $1 - 1/\exp 1$.

```

definition greedy_ratio :: real where
  "greedy_ratio = (if OPT_k = 0 then 1 else f (greedy_set k) / OPT_k)"

```

```

corollary greedy_ratio_ge_1_minus_inv_exp:
  assumes "k > 0"
  shows "greedy_ratio ≥ 1 - 1 / exp 1"

```

<proof>

end

7 Step-spec corollary

Any oracle satisfying the step-spec assumptions inherits the Nemhauser–Wolsey approximation guarantee immediately via the sublocale from *Greedy_Step_Oracle* to *Greedy_Setup*.

context *Greedy_Step_Oracle*
begin

theorem *greedy_step_oracle_approximation*:
 assumes "k > 0"
 shows
 " f (*Greedy_Setup.greedy_set* V *select* k)
 $\geq (1 - 1 / \exp 1) * \text{Greedy_Setup.OPT_k V f k}$ "
 <proof>

end

end

theory *Lazy_Greedy_Stateful_StepSpec*
 imports "../Algorithms/Lazy_Greedy_Stateful"
begin

Sequence-level lemmas for the verified stateful lazy run.

This theory packages the per-iteration facts needed by the approximation proof, such as the membership and maximal-gain properties of the chosen lazy element at step *i*, together with the update equation for the next lazy set.

It is not an instance of the stateless greedy step oracle locale. Instead, it exposes properties of the concrete verified run.

context *Cardinality_Constraint*
begin

abbreviation *st_i* :: "nat $\Rightarrow$ 'a lg_state" **where**
 "*st_i* i $\equiv$ *lazy_state* i"

abbreviation *S_i* :: "nat $\Rightarrow$ 'a set" **where**
 "*S_i* i $\equiv$ *lazy_set* i"

abbreviation *A_i* :: "nat $\Rightarrow$ 'a set" **where**
 "*A_i* i $\equiv$ *remaining* (*st_i* i)"

lemma *A_i_eq*: "*A_i* i = V - *S_i* i"
 <proof>

```

definition lazy_choice :: "nat  $\Rightarrow$  'a" where
  "lazy_choice i = fst (lazy_select (S_i i) (A_i i) (ubg (st_i i)))"

lemma A_i_finite: "finite (A_i i)"
   $\langle$ proof $\rangle$ 

lemma lazy_choice_mem:
  assumes ne: "A_i i  $\neq$  {}"
  shows "lazy_choice i  $\in$  A_i i"
   $\langle$ proof $\rangle$ 

lemma lazy_choice_max:
  assumes ne: "A_i i  $\neq$  {}"
  shows " $\forall y \in A_i i. \text{gain } (S_i i) y \leq \text{gain } (S_i i) (\text{lazy\_choice } i)$ "
   $\langle$ proof $\rangle$ 

lemma lazy_set_Suc_insert:
  assumes ne: "A_i i  $\neq$  {}"
  shows "S_i (Suc i) = insert (lazy_choice i) (S_i i)"
   $\langle$ proof $\rangle$ 

lemma lazy_choice_in_V_minus_S:
  assumes "V - lazy_set i  $\neq$  {}"
  shows "lazy_choice i  $\in$  V - lazy_set i"
   $\langle$ proof $\rangle$ 

lemma lazy_choice_argmax_V_minus_S:
  assumes "V - lazy_set i  $\neq$  {}"
  shows " $\forall y \in V - \text{lazy\_set } i. \text{gain } (\text{lazy\_set } i) y \leq \text{gain } (\text{lazy\_set } i) (\text{lazy\_choice } i)$ "
   $\langle$ proof $\rangle$ 

lemma lazy_set_Suc_insert_V_minus_S:
  assumes "V - lazy_set i  $\neq$  {}"
  shows "lazy_set (Suc i) = insert (lazy_choice i) (lazy_set i)"
   $\langle$ proof $\rangle$ 

end
end
theory Lazy_Greedy_Stateful_Approx
imports
  Greedy_Submodular_Approx
  Lazy_Greedy_Stateful_StepSpec
begin

```

Approximation guarantee for the verified stateful lazy greedy construction.

This theory treats the stateful lazy algorithm as an implementation-

level refinement. It reuses the optimal-value infrastructure from the greedy approximation development, together with the per-iteration lemmas from the lazy step-spec theory, and proves a corresponding gap recurrence for the lazy construction.

In particular, this theory does not instantiate the stateless step-spec locale. Instead, it works directly with the verified lazy run and its sequence-level properties.

```
context Cardinality_Constraint
begin
```

```
interpretation Greedy_Base: Greedy_Setup V f k argmax_gain_some
  ⟨proof⟩
```

```
abbreviation OPT_k :: real where
  "OPT_k ≡ Greedy_Base.OPT_k"
```

```
definition gapL :: "nat ⇒ real" where
  "gapL i = OPT_k - f (lazy_set i)"
```

```
lemma lazy_set_0[simp]: "lazy_set 0 = {}"
  ⟨proof⟩
```

```
lemma lazy_set_subset_V[simp]: "lazy_set i ⊆ V"
  ⟨proof⟩
```

```
lemma lazy_set_finite[simp]: "finite (lazy_set i)"
  ⟨proof⟩
```

```
lemma remaining_lazy_state[simp]: "remaining (lazy_state i) = V - lazy_set i"
  ⟨proof⟩
```

```
lemma card_lazy_le_i: "card (lazy_set i) ≤ i"
  ⟨proof⟩
```

```
lemma card_lazy_lt_k:
  "i < k ⇒ card (lazy_set i) < k"
  ⟨proof⟩
```

```
lemma lazy_remainder_nonempty:
  "i < k ⇒ V - lazy_set i ≠ {}"
  ⟨proof⟩
```

```
lemma lazy_set_feasible:
  assumes "i ≤ k"
  shows "feasible (lazy_set i)"
  ⟨proof⟩
```

```

lemma gapL_nonneg:
  assumes "i ≤ k"
  shows "0 ≤ gapL i"
  <proof>

lemma lazy_step_ineq:
  "i < k ⇒ gain (lazy_set i) (lazy_choice i) ≥ gapL i / real k"
  <proof>

lemma gapL_step:
  "i < k ⇒ gapL (Suc i) ≤ (1 - 1 / real k) * gapL i"
  <proof>

lemma gapL_geometric:
  "k > 0 ⇒ i ≤ k ⇒ gapL i ≤ (1 - 1 / real k) ^ i * OPT_k"
  <proof>

theorem lazy_stateful_approximation:
  assumes k_pos: "k > 0"
  shows "f (lazy_set k) ≥ (1 - 1 / exp 1) * OPT_k"
  <proof>

end

end

```

References

- [1] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14(1):265–294, 1978.